

Relatório — Compartilhamento de Recursos: Produtor e Consumidor (Padaria)

Nome

Felipe Beserra de Oliveira

Número USP

13683702

Respostas, comentários e análises

Descrição da Atividade

O experimento simula uma padaria utilizando duas threads do Zephyr RTOS:

- **Padeiro (Thread 1):** produz um pão a cada 1 segundo e o coloca na vitrine.
- **Cliente (Thread 2):** tenta retirar um pão da vitrine a cada 1,5 segundos.

A vitrine é representada pela variável compartilhada `saldo_vitrine`:

- Produção: `saldo_vitrine++`
- Consumo: `saldo_vitrine--`

A atividade é dividida em três partes, cada uma implementada como uma biblioteca separada em `lib/`, selecionável em `src/main.c` através da constante `PADARIA_PARTE`:

Parte	Biblioteca	Mecanismo
1	<code>lib/padaria_p1</code>	Sem sincronização
2	<code>lib/padaria_p2</code>	Mutex
3	<code>lib/padaria_p3</code>	Semáforos (mutex + 2 semáforos)

Parte 1 — Sem sincronização

Regras:

- O padeiro produz um pão a cada 1 segundo (`saldo_vitrine++`).
- O cliente tenta retirar um pão a cada 1,5 segundos (`saldo_vitrine--`).
- Nenhum mutex, semáforo ou fila é utilizado.

Log obtido (`logs/P1-COM8_2026_06_12.11.03.59.369.txt`):

```
=== Padaria - Parte 1 (sem sincronizacao) ===
[Padeiro] produziu pao -> saldo_vitrine = 1
[Cliente] retirou pao -> saldo_vitrine = 0
[Padeiro] produziu pao -> saldo_vitrine = 1
[Cliente] retirou pao -> saldo_vitrine = 0
...
[Padeiro] produziu pao -> saldo_vitrine = 20
[Cliente] retirou pao -> saldo_vitrine = 19
[Padeiro] produziu pao -> saldo_vitrine = 20
[Cliente] retirou pao -> saldo_vitrine = 19
[Padeiro] produziu pao -> saldo_vitrine = 20
[Padeiro] produziu pao -> saldo_vitrine = 21
[Cliente] retirou pao -> saldo_vitrine = 20
[Padeiro] produziu pao -> saldo_vitrine = 21
```

Perguntas de observação (executado com `PADARIA_PARTE 1`):

1. O saldo da vitrine permanece consistente?
 - Nesta execução sim: cada `produziu` aumenta o valor em 1 e cada `retirou` diminui em 1, sem saltos ou valores repetidos indevidamente. Isso ocorre porque o padeiro (1 s) e o cliente (1,5 s) raramente são escalonados no mesmo instante, então o `saldo_vitrine++/--` (que não é atômico — envolve load, soma/subtração e store) quase nunca é interrompido no meio. Essa consistência é, portanto, fruto da temporização escolhida, e não de uma garantia do código.
2. O saldo pode se tornar negativo?
 - Não, nesta execução. Como o padeiro produz mais rápido (1 s) do que o cliente consome (1,5 s), o saldo cresce continuamente (0→1→0→1→1→2→...→21) e nunca se aproxima de zero após o início, logo o cliente nunca encontra `saldo_vitrine == 0`.
3. O comportamento muda ao alterar os tempos das threads?
 - Sim. Se o intervalo do cliente fosse menor que o do padeiro (cliente mais rápido), o saldo tenderia a cair e poderia ficar negativo, já que o cliente sempre executa `saldo_vitrine--` sem verificar se há pão disponível. Além disso, com intervalos muito próximos (alta frequência de acesso), aumenta a chance real de as duas threads serem escalonadas exatamente durante a leitura/escrita de `saldo_vitrine`, gerando perda de atualizações (race condition).
4. O comportamento muda ao alterar as prioridades das threads?
 - No código atual ambas as threads têm a mesma prioridade (5) e cada uma libera a CPU ao chamar `k_sleep/k_msleep`, então a prioridade pouco influencia. Se uma das threads tivesse prioridade muito mais alta e um período de execução comparável, ela poderia preemptar a outra exatamente no meio do `saldo_vitrine++/--`, aumentando a probabilidade de inconsistências — mesmo que neste teste isso não tenha sido observado.
5. Quais problemas podem ocorrer quando duas threads acessam a mesma variável simultaneamente?
 - Condição de corrida (*race condition*): se as duas threads lerem o valor antigo de `saldo_vitrine` antes que a outra grave o novo valor, uma das atualizações é perdida (*lost update*). O valor final fica menor (ou maior) do que o esperado pela soma das produções e consumos, e o resultado passa a depender da ordem de escalonamento (comportamento não determinístico).

Um mutex (`vitrine_mutex`) protege todo acesso (leitura e escrita) à variável `saldo_vitrine`, garantindo que o incremento/decremento seja uma operação atômica.

Log obtido (`logs/P2-COM8_2026_06_12.11.17.21.663.txt`):

```
=== Padaria - Parte 2 (com mutex) ===
[Padeiro] produziu pao -> saldo_vitrine = 1
[Cliente] retirou pao -> saldo_vitrine = 0
[Padeiro] produziu pao -> saldo_vitrine = 1
...
[Padeiro] produziu pao -> saldo_vitrine = 20
[Padeiro] produziu pao -> saldo_vitrine = 21
[Cliente] retirou pao -> saldo_vitrine = 20
```

Perguntas de observação (executado com `PADARIA PARTE 2`):

1. O mutex eliminou os problemas observados na Parte 1?
 - O log da Parte 2 é praticamente idêntico ao da Parte 1 (mesma sequência de valores, mesmo crescimento de 0 até ~21), porque na Parte 1, nesta velocidade, o problema de race condition também não havia se manifestado visivelmente. O mutex elimina a *causa* do problema (garante que `saldo_vitrine++/--` seja atômico, sem leitura/escrita intercaladas), então o saldo continua numericamente correto mesmo que as threads sejam escalonadas em momentos críticos — algo que a Parte 1 não garantia.
2. O mutex impede que o cliente retire pão de uma vitrine vazia?
 - Não. O `cliente_thread` continua executando `saldo_vitrine--` incondicionalmente, apenas protegido pelo mutex — não há nenhuma verificação `if (saldo_vitrine > 0)`. Se o cliente fosse mais rápido que o padeiro, o saldo chegaria a 0 e o mutex permitiria, ainda assim, que `saldo_vitrine` se tornasse -1, -2, etc. O mutex garante apenas que a operação seja atômica, não que ela seja válida.
3. Qual é exatamente o recurso protegido pelo mutex?
 - A variável compartilhada `saldo_vitrine` — mais precisamente, a seção crítica que lê o valor atual, soma/subtrai 1 e grava o novo valor (incluindo o `printf` que o exibe).
4. Qual é a principal função de um mutex neste problema?
 - Garantir exclusão mútua: apenas uma thread por vez pode executar a seção crítica de leitura-modificação-escrita de `saldo_vitrine`, evitando que o padeiro e o cliente atualizem a variável "ao mesmo tempo" e causem perda de atualizações (*lost update*).

Parte 3 — Utilizando Semáforos

A vitrine passa a ter capacidade máxima de **10 pães**. Dois semáforos de contagem controlam a disponibilidade do recurso, além do mutex que protege `saldo_vitrine`:

- `sem_paes` (inicial 0, máx. 10): número de pães disponíveis. O cliente faz `k_sem_take(&sem_paes)` antes de retirar — bloqueia se não houver pão.
- `sem_espaco` (inicial 10, máx. 10): número de vagas livres na vitrine. O padeiro faz `k_sem_take(&sem_espaco)` antes de produzir — bloqueia se a vitrine estiver cheia.

Log obtido (logs/P3-COM8_2026_06_12.11.22.55.992.txt):

```
=== Padaria - Parte 3 (com semaforos, capacidade = 10) ===
[Padeiro] produziu pao -> saldo_vitrine = 1
[Cliente] retirou pao -> saldo_vitrine = 0
...
[Padeiro] produziu pao -> saldo_vitrine = 9
[Padeiro] produziu pao -> saldo_vitrine = 10
[Cliente] retirou pao -> saldo_vitrine = 9
[Padeiro] produziu pao -> saldo_vitrine = 10
[Cliente] retirou pao -> saldo_vitrine = 9
[Padeiro] produziu pao -> saldo_vitrine = 10
[Cliente] retirou pao -> saldo_vitrine = 9
... (oscila indefinidamente entre 9 e 10)
```

Perguntas de observação (executado com *PADARIA_PARTE 3*):

1. O saldo pode se tornar negativo?
 - Não. *sem_paes* é inicializado com 0, então o cliente bloqueia em *k_sem_take(&sem_paes, K_FOREVER)* sempre que não há pão produzido, só conseguindo decrementar *saldo_vitrine* depois que o padeiro sinalizar *k_sem_give(&sem_paes)*. No log, o valor mínimo observado é 0.
2. O saldo pode ultrapassar a capacidade da vitrine?
 - Não. *sem_espaco* é inicializado com *VITRINE_CAPACIDADE = 10*, então o padeiro bloqueia em *k_sem_take(&sem_espaco, K_FOREVER)* quando a vitrine está cheia. No log, depois que *saldo_vitrine* atinge 10, o padeiro passa a ficar bloqueado até o cliente retirar um pão (*k_sem_give(&sem_espaco)*), e o sistema entra em regime permanente oscilando apenas entre 9 e 10 — nunca chega a 11.
3. Qual problema foi resolvido pelos semáforos que não era resolvido apenas pelo mutex?
 - O controle de **disponibilidade/capacidade** do recurso. O mutex (Parte 2) só garantia que o acesso a *saldo_vitrine* fosse atômico, mas não impedia o cliente de retirar de uma vitrine vazia nem o padeiro de produzir além da capacidade. Os semáforos fazem as threads **bloquearem** (dormirem) até que a condição lógica (há pão / há espaço) seja satisfeita.
4. Qual é o papel de cada semáforo utilizado?
 - *sem_paes* (0..10): representa a quantidade de pães disponíveis na vitrine. O padeiro dá *k_sem_give* após produzir; o cliente faz *k_sem_take* antes de retirar (bloqueia se for 0).
 - *sem_espaco* (0..10, inicia em 10): representa as vagas livres na vitrine. O cliente dá *k_sem_give* após retirar um pão; o padeiro faz *k_sem_take* antes de produzir (bloqueia se for 0, ou seja, vitrine cheia).

Comparação

1. Qual a diferença entre proteger um recurso e controlar sua disponibilidade?
 - Proteger um recurso (mutex) significa garantir que o acesso à variável compartilhada seja exclusivo e atômico, evitando que duas threads leiam/escrevam o mesmo dado ao mesmo tempo e corrompam seu valor. Controlar a disponibilidade (semáforo) significa garantir que uma operação só ocorra quando existir quantidade suficiente do recurso (pão para retirar, espaço

para produzir), bloqueando a thread até que essa condição seja verdadeira. Uma coisa é "o dado não pode ser corrompido"; outra é "a operação não pode ser realizada agora".

2. Em qual parte da atividade o mutex foi suficiente?

- Na Parte 2, o mutex foi suficiente apenas para o objetivo restrito de proteger a consistência aritmética de `saldo_vitrine` (evitar *lost updates*). Ele não foi suficiente para impedir saldo negativo nem para limitar a capacidade da vitrine — esses problemas continuam presentes mesmo com o mutex.

3. Em qual parte os semáforos se mostraram mais adequados?

- Na Parte 3, pois resolveram exatamente os problemas de disponibilidade/capacidade: o cliente nunca retira de uma vitrine vazia e o padeiro nunca excede a capacidade de 10 pães, como confirmado pelo log (oscilação estável entre 9 e 10).

4. Se fosse necessário implementar uma padaria real, você utilizaria mutex, semáforos ou ambos?

Justifique.

- Ambos, como na Parte 3. Os semáforos (`sem_paes/sem_espaco`) controlam a disponibilidade do recurso — bloqueando padeiro e cliente conforme a vitrine está vazia ou cheia — enquanto o mutex protege a atualização da variável `saldo_vitrine` em si, garantindo que a contagem permaneça correta mesmo quando ambas as condições de semáforo são satisfeitas "ao mesmo tempo". Usar apenas mutex não resolve o controle de disponibilidade, e usar apenas semáforos não garante, por si só, a atomicidade da leitura-modificação-escrita da variável compartilhada.

Código (main.c)

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include <padaria_p1.h>
#include <padaria_p2.h>
#include <padaria_p3.h>

/*
 * Seleccione qual parte da atividade executar:
 * 1 -> Parte 1: padeiro e cliente sem sincronizacao
 * 2 -> Parte 2: padeiro e cliente com mutex
 * 3 -> Parte 3: padeiro e cliente com semaforos (capacidade da vitrine)
 */
#define PADARIA_PARTE 1

int main(void)
{
    #if PADARIA_PARTE == 1
        padaria_p1_start();
    #elif PADARIA_PARTE == 2
        padaria_p2_start();
    #elif PADARIA_PARTE == 3
        padaria_p3_start();
    #else
        #error "PADARIA_PARTE deve ser 1, 2 ou 3"
    #endif
}
```

```
    return 0;
}
```

Código da Parte 1 — lib/padaria_p1/padaria_p1.c (sem sincronização)

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include "padaria_p1.h"

#define STACK_SIZE 1024
#define THREAD_PRIORITY 5

/* Variavel compartilhada SEM protecao (mutex/semaforo) de propósito */
static int saldo_vitrine;

K_THREAD_STACK_DEFINE(padeiro_p1_stack, STACK_SIZE);
K_THREAD_STACK_DEFINE(cliente_p1_stack, STACK_SIZE);
static struct k_thread padeiro_p1_thread;
static struct k_thread cliente_p1_thread;

static void padeiro_thread(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        saldo_vitrine++;
        printk("[Padeiro] produziu pao -> saldo_vitrine = %d\n", saldo_vitrine);
        k_sleep(K_SECONDS(1));
    }
}

static void cliente_thread(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        saldo_vitrine--;
        printk("[Cliente] retirou pao -> saldo_vitrine = %d\n", saldo_vitrine);
        k_msleep(1500);
    }
}

void padaria_p1_start(void)
{
    saldo_vitrine = 0;

    printk("=== Padaria - Parte 1 (sem sincronizacao) ===\n");
}
```

```
k_thread_create(&padeiro_p1_thread, padeiro_p1_stack, STACK_SIZE,
                padeiro_thread, NULL, NULL, NULL,
                THREAD_PRIORITY, 0, K_NO_WAIT);

k_thread_create(&cliente_p1_thread, cliente_p1_stack, STACK_SIZE,
                cliente_thread, NULL, NULL, NULL,
                THREAD_PRIORITY, 0, K_NO_WAIT);

}
```

Código da Parte 2 — `lib/padaria_p2/padaria_p2.c` (com mutex)

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include "padaria_p2.h"

#define STACK_SIZE 1024
#define THREAD_PRIORITY 5

/* Variavel compartilhada protegida por mutex */
static int saldo_vitrine;
static struct k_mutex vitrine_mutex;

K_THREAD_STACK_DEFINE(padeiro_p2_stack, STACK_SIZE);
K_THREAD_STACK_DEFINE(cliente_p2_stack, STACK_SIZE);
static struct k_thread padeiro_p2_thread;
static struct k_thread cliente_p2_thread;

static void padeiro_thread(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        k_mutex_lock(&vitrine_mutex, K_FOREVER);
        saldo_vitrine++;
        printk("[Padeiro] produziu pao -> saldo_vitrine = %d\n", saldo_vitrine);
        k_mutex_unlock(&vitrine_mutex);

        k_sleep(K_SECONDS(1));
    }
}

static void cliente_thread(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
```

```

        k_mutex_lock(&vitrine_mutex, K_FOREVER);
        saldo_vitrine--;
        printk("[Cliente] retirou pao -> saldo_vitrine = %d\n", saldo_vitrine);
        k_mutex_unlock(&vitrine_mutex);

        k_msleep(1500);
    }
}

void padaria_p2_start(void)
{
    saldo_vitrine = 0;
    k_mutex_init(&vitrine_mutex);

    printk("=== Padaria - Parte 2 (com mutex) ===\n");

    k_thread_create(&padeiro_p2_thread, padeiro_p2_stack, STACK_SIZE,
                    padeiro_thread, NULL, NULL, NULL,
                    THREAD_PRIORITY, 0, K_NO_WAIT);

    k_thread_create(&cliente_p2_thread, cliente_p2_stack, STACK_SIZE,
                    cliente_thread, NULL, NULL, NULL,
                    THREAD_PRIORITY, 0, K_NO_WAIT);
}

```

Código da Parte 3 — `lib/padaria_p3/padaria_p3.c` (com semáforos)

```

#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include "padaria_p3.h"

#define STACK_SIZE 1024
#define THREAD_PRIORITY 5

#define VITRINE_CAPACIDADE 10

/* Variavel compartilhada protegida por mutex + sincronizada por semaforos */
static int saldo_vitrine;
static struct k_mutex vitrine_mutex;

/* sem_paes:  conta paes disponiveis na vitrine  (0..VITRINE_CAPACIDADE) */
/* sem_espaco: conta espacos livres na vitrine  (0..VITRINE_CAPACIDADE) */
static struct k_sem sem_paes;
static struct k_sem sem_espaco;

K_THREAD_STACK_DEFINE(padeiro_p3_stack, STACK_SIZE);
K_THREAD_STACK_DEFINE(cliente_p3_stack, STACK_SIZE);
static struct k_thread padeiro_p3_thread;
static struct k_thread cliente_p3_thread;

static void padeiro_thread(void *p1, void *p2, void *p3)

```



```
{
    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        /* Aguarda haver espaco livre na vitrine */
        k_sem_take(&sem_espaco, K_FOREVER);

        k_mutex_lock(&vitrine_mutex, K_FOREVER);
        saldo_vitrine++;
        printk("[Padeiro] produziu pao -> saldo_vitrine = %d\n", saldo_vitrine);
        k_mutex_unlock(&vitrine_mutex);

        /* Sinaliza que ha mais um pao disponivel */
        k_sem_give(&sem_paes);

        k_sleep(K_SECONDS(1));
    }
}

static void cliente_thread(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        /* Aguarda haver pao disponivel na vitrine */
        k_sem_take(&sem_paes, K_FOREVER);

        k_mutex_lock(&vitrine_mutex, K_FOREVER);
        saldo_vitrine--;
        printk("[Cliente] retirou pao -> saldo_vitrine = %d\n", saldo_vitrine);
        k_mutex_unlock(&vitrine_mutex);

        /* Sinaliza que ha mais um espaco livre na vitrine */
        k_sem_give(&sem_espaco);

        k_msleep(1500);
    }
}

void padaria_p3_start(void)
{
    saldo_vitrine = 0;
    k_mutex_init(&vitrine_mutex);

    /* Vitrine comeca vazia: nenhum pao disponivel, capacidade toda livre */
    k_sem_init(&sem_paes, 0, VITRINE_CAPACIDADE);
    k_sem_init(&sem_espaco, VITRINE_CAPACIDADE, VITRINE_CAPACIDADE);

    printk("=== Padaria - Parte 3 (com semaforos, capacidade = %d) ===\n",
           VITRINE_CAPACIDADE);
}
```

```
k_thread_create(&padeiro_p3_thread, padeiro_p3_stack, STACK_SIZE,  
                padeiro_thread, NULL, NULL, NULL,  
                THREAD_PRIORITY, 0, K_NO_WAIT);  
  
k_thread_create(&cliente_p3_thread, cliente_p3_stack, STACK_SIZE,  
                cliente_thread, NULL, NULL, NULL,  
                THREAD_PRIORITY, 0, K_NO_WAIT);  
  
}
```

Repositório

<https://github.com/Beserrovsky/Atividade-7>